ET4351 Lab 1
Project Baseline with `PicoSoC`

1 Revision History

Date of Revision	Contributor	Email
May 2023	Chang Gao	chang.gao@tudelft.nl
April 2024	Nicolas Chauvaux	n.chauvaux@tudelft.nl
	Douwe den Blanken	d.m.j.denblanken@tudelft.nl
Jan 2025	Ang Li	ang.li@tudelft.nl
	Yizhuo Wu	yizhuo.wu@tudelft.nl
Jan 2026	Nicolas Chauvaux	n.chauvaux@tudelft.nl
	Douwe den Blanken	d.m.j.denblanken@tudelft.nl
	Guilherme Guedes	g.guedes@tudelft.nl

2 Introduction

In this laboratory exercise, you will gain a comprehensive understanding of the following topics:

- The fundamentals of Reduced Instruction Set Computer Five (**RISC-V**), an open-source Instruction Set Architecture (**ISA**) for computer processors.
- The simulation of a simple System-on-Chip (**SoC**) and how to personalize it for your needs by integrating a hardware accelerator coded in Verilog.
- The process of accessing a remote server through Secure Shell Protocol (**SSH**).
- The simulation of a basic digital circuit using `QuestaSim`[1], a tool developed by `Siemens EDA` (formerly `Mentor Graphics`).
- The compilation of C code using the RISC-V cross-compiler.
- The execution of your compiled C code on the **PicoRV32** core.
- The creation of a dummy accelerator connected to the **PicoSoC** and its interaction with the PicoRV32 processor that executes a C program.
- The implementation of automation using **Makefiles** and **Bash Scripts**.

[1] `https://eda.sw.siemens.com/en-US/ic/questa/simulation/advanced-simulator/`

For questions on the labs and project, please contact **Douwe den Blanken** (d.m.j.denblanken@tudelft.nl), **Nicolas Chauvaux** (n.chauvaux@tudelft.nl), **Pepijn Kremers** (p.kremers@tudelft.nl), and **Yizhuo Wu** (yizhuo.wu@tudelft.nl).

2.1 RISC-V

RISC-V, the open source ISA, is rapidly gaining popularity in the computing industry by being simple and flexible. RISC-V eliminates licensing fees as an open-source project, making it an attractive alternative to proprietary ISAs like Advanced RISC Machines (**ARM**) and x86. This has led to a thriving community of developers, researchers, and businesses collaborating on developing RISC-V-based hardware, software, and development tools[2].

The modular nature of RISC-V allows for a high degree of customization, enabling designers to create processors tailored to specific applications or requirements. This flexibility has attracted interest from various industries, including IoT, data center infrastructure, automotive, and aerospace. The RISC-V ecosystem also benefits from a robust software infrastructure, with support from popular compilers like GCC[3] and open-source operating systems such as Linux.

One of the key factors driving RISC-V adoption is its potential to democratize the processor industry, providing a level playing field for small and large companies. According to Deloitte Global[4], the market for RISC-V processing cores is predicted to double in 2022 from what it was in 2021 and double again in 2023 as the market for RISC-V processing cores continues to expand, as depicted in Figure 1. By lowering the barriers to entry, RISC-V empowers startups and smaller organizations to innovate and compete with established players in the market. This competitive landscape fosters rapid innovation, developing more efficient, powerful, and cost-effective computing solutions.

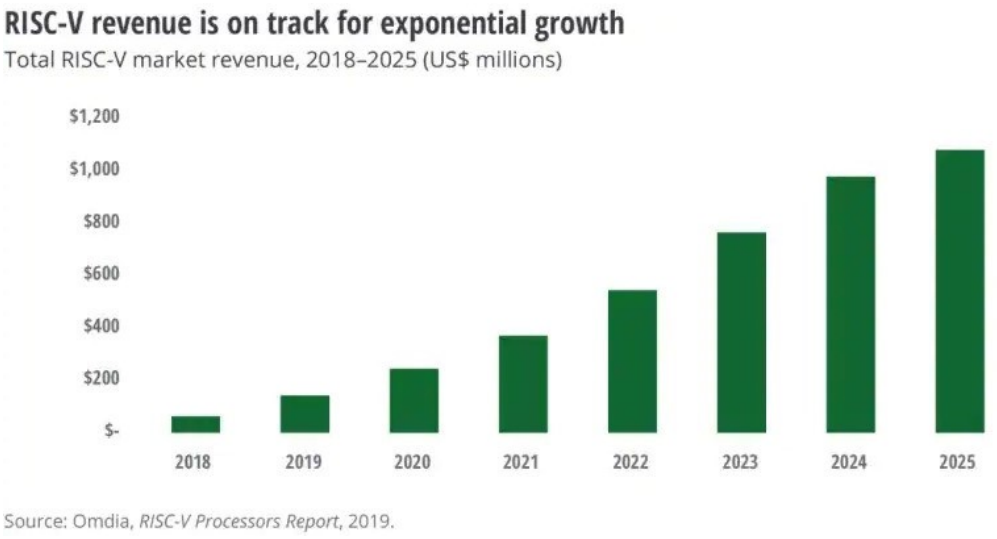

Figure 1: Market trend of RISC-V

RISC-V is a groundbreaking open-source ISA that has the potential to reshape the computing landscape. The open licensing, vibrant ecosystem, and focus on real-world applicability of RISC-

[2]RISC-V Specifications: `https://riscv.org/technical/specifications/`

[3]The GCC Compiler: `https://gcc.gnu.org/`

[4]Source: https://www2.deloitte.com/us/en/insights/industry/technology/technology-media-and-telecom-predictions/2022/risc-v-open-source-cpu.html

V make it an attractive option for businesses and developers alike. As the RISC-V community continues to grow and innovate, it is poised to play a significant role in the future of the computing industry, offering a compelling alternative to traditional proprietary ISAs.

2.2 PicoRV32

PicoRV32 is a compact, open-source CPU core that implements the RISC-V RV32IMC ISA[5]. The core can be configured as RV32E, RV32I, RV32IC, RV32IM, or RV32IMC[6], with optional built-in interrupt controller support. PicoRV32 is designed for FPGA and ASIC implementations, typically serving as an auxiliary processor in bigger systems.

The PicoRV32 source code is concise and well-organized, making it an ideal educational resource for students to learn how a simplistic RISC-V CPU can be modeled using the Verilog language.

2.3 PicoSoC

An SoC is an integrated circuit (**IC**) that integrates multiple components, such as microprocessors, memory, input/output interfaces, onto a single chip. This allows for smaller, more efficient, and less expensive devices than systems composed of multiple chips to perform the same functions. SoCs are widely used in various devices, including smartphones, tablets, laptops, and Internet-of-Things (**IoT**) devices.

PicoSoC[7] is a straightforward, easy-to-use example SoC design based on the PicoRV32 CPU core. It is designed to run code directly from an off-chip SPI flash memory, serving as an out-of-the-box solution for simple control tasks in ASIC and FPGA designs.

The block diagram of PicoSoC is shown in Fig. 2.

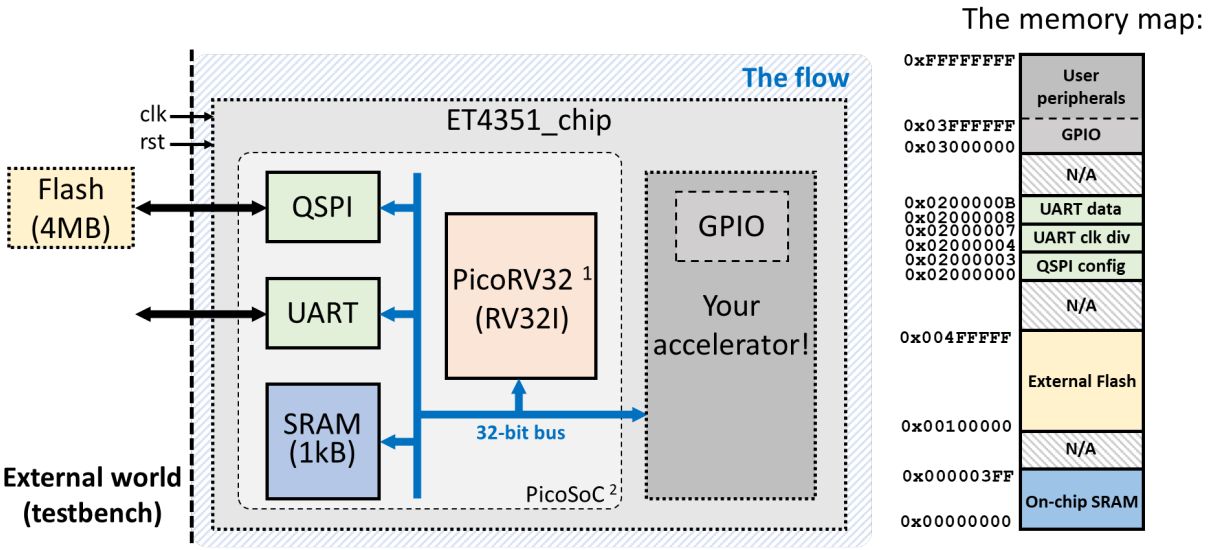

Figure 2: Block diagram of `PicoSoC` and the memory map.

All peripherals are memory-mapped (i.e. to access these, you access a fixed address on the communication bus subsystem).

[5] `https://github.com/YosysHQ/picorv32`

[6] The 'RV32' part indicates a RISC-V 32-bit architecture and the letters that follow denote extensions, e.g., 'I': Integer, 'M': Multiplication and division, 'C': Compressed, 'E': Embedded.

[7] `https://github.com/YosysHQ/picorv32/tree/master/picosoc`

3 Tutorial 1: Connecting to the server

One can connect to the server using SSH, which can be combined with `X11 Server` to stream the Graphical User Interface (**GUI**) running on the server to the desktop of your local machine. However, current server resources do not allow for all students to individually connect and execute computationally expensive tasks at the same time. Therefore, each group should connect to the server by using **only one** laptop).

3.1 Setup SSH & X11 Server

How you use SSH and `X11 Server` highly depends on your Operating System (**OS**).

3.1.1 Windows

If you have a Windows PC, please download the installer version of `MobaXterm` (`https://mobaxterm.mobatek.net/download-home-edition.html`). `MobaXterm` is a terminal software for Windows that integrates `Unix` tools, `X11 Server`, and network tools for remote computing into one interface. It is designed for system administrators and network engineers.

After installation, open `MobaXterm` and click `Session` as shown in Figure 3.

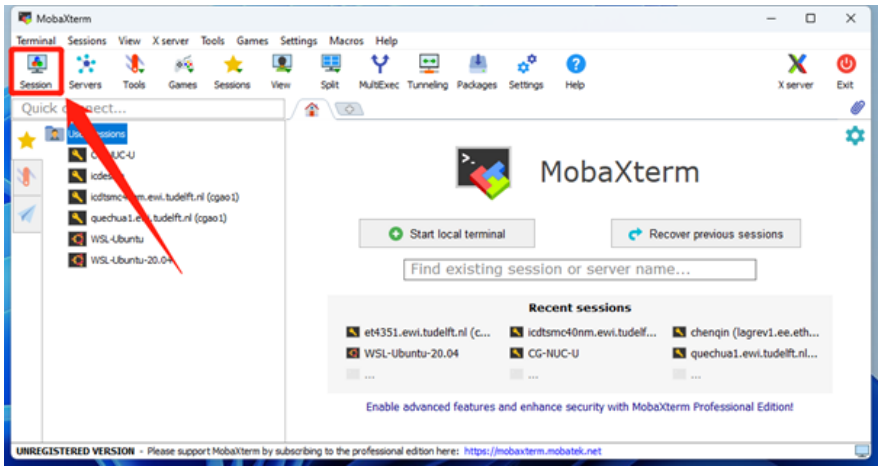

Figure 3: `MobaXterm` Step 1

In the new window "Session Settings" shown in Figure 4:

1. Click `SSH`.
2. Type in the address of the server `et4351.ewi.tudelft.nl`.
3. Type in your NetID.

You will receive a terminal prompt asking you to type in your password. If you type in your password correctly, you will now be connected to the remote server, as shown in Figure 5:

3.1.2 Linux

If you use a Linux distribution such as Ubuntu, you can use the `ssh` command directly in a terminal. You can open a terminal by using the keyboard shortcut `Ctrl + Alt + T`. Then you can use the following command to connect to any remote host server:

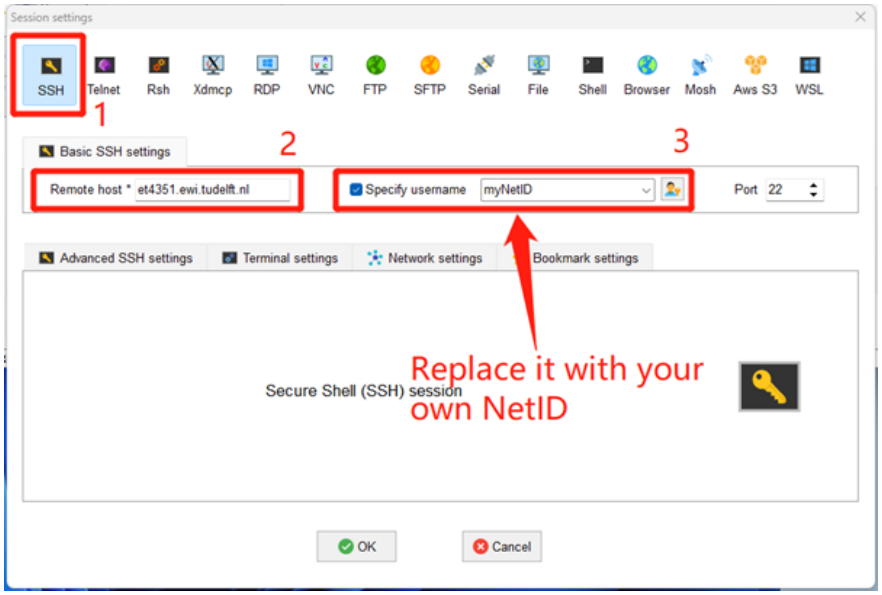

Figure 4: `MobaXterm` Step 2

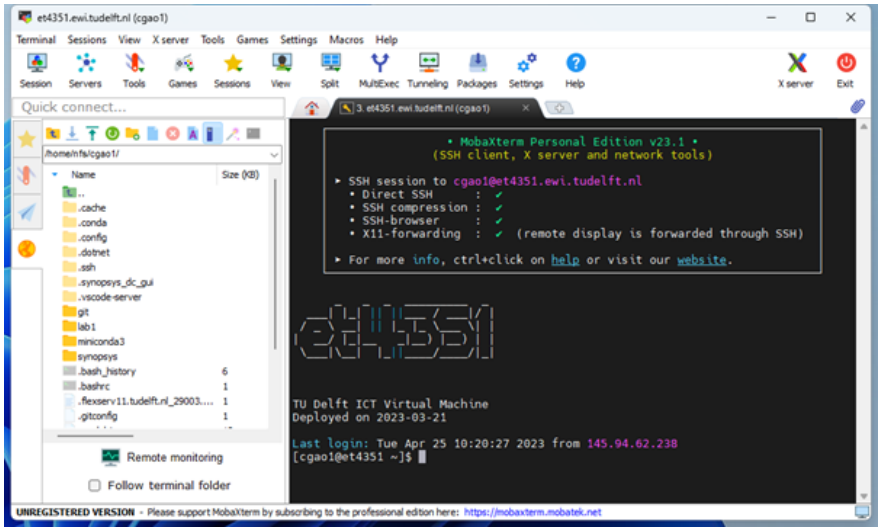

Figure 5: `MobaXterm` connected to the remote server

```
ssh -X <username>@<server_address>
```

The `-X` option turns on the X11 Server and allows the programs' GUI to show up on your PC when launching a command. In our case, `server_address` is `et4351.ewi.tudelft.nl`, while `username` is your **NetID**. Type the following command in your terminal:

```
ssh -X myNetID@et4351.ewi.tudelft.nl
```

Then you will see the following terminal prompts:

```
myNetID@et4351.ewi.tudelft.nl's password:
```

Type in the password associated with your **NetID** and press `Enter`. The password will not be printed on the terminal during your typing. Then you should have successfully logged into the `et4351.ewi.tudelft.nl` and your terminal will show something like below:

```
[myNetID@et4351 ~]$
```

The text `[myNetID@et4351 ~]$` displayed in the terminal indicates the system is ready to receive and process commands. It typically displays the `username`, `hostname`, the current working directory, and prompts the user to enter commands. The symbol `~` stands for the path to your home directory. To know the absolute path of your home directory, try the `pwd` command.

3.1.3 macOS

To enable X11 forwarding on `macOS`, you must install `XQuartz`, an open-source implementation of the X Window System that runs on macOS. It provides the X11 server required to run GUI programs remotely. Download the latest version from the official website (`https://www.xquartz.org`). Once the download is complete, open the DMG file and follow the instructions to install `XQuartz` on your Mac.

After the installation, launch `XQuartz` from your `LaunchPad` and then open the `Terminal` app on your Mac and run the same command in Linux to connect to the remote server:

```
ssh -X <username>@<server_address>
```

3.2 Access Remote Files

To conveniently access remote files, you can use dedicated SFTP clients like WinSCP for Windows and FileZilla for Windows, Linux, and macOS. This brief tutorial will provide download links and encourage exploring these clients. Though MobaXterm also has a built-in SFTP client, you may find these dedicated clients easier to use.

Choose your SFTP client and explore:

- WinSCP (Windows): `https://winscp.net/eng/download.php`
- FileZilla (Windows, Linux, macOS): `https://filezilla-project.org/download.php?type=client`

*Please do not use the `Remote SSH` function of `Visual Studio Code` since it will utilize around 1 GB RAM per user. At the same time, the `et4351` server has very limited computational and memory resources.

4 Tutorial 2: Simulation with `QuestaSim`

4.1 Step-by-Step

Run the following commands to copy the lab files to your home directory.

```
cd ~
cp -r /data/labs/2026/labs/lab1 ~/
cd lab1
source setup.sh
cd tut2
```

`setup.sh` is a `Bash Script` that automatically setup the lab environment. You must execute `source setup.sh` so that the Electronic Design Automation (**EDA**) tools can locate their license. We will explain what `Bash Script` is in the next section. Also, you navigated to the `tut2` folder in our lab 1 directory.

A `SystemVerilog` design of a 32-bit integer multiplier, named `mul.sv`, is located under `~/lab1/tut2/src/design`.

```
module multiplier_32bit #(
    parameter A_WIDTH = 32,
    parameter B_WIDTH = 32,
    parameter C_WIDTH = 64
    )(
    input  logic         clk,
    input  logic         resetn,
    input  logic  [A_WIDTH-1:0] a,
    input  logic  [B_WIDTH-1:0] b,
    output logic  [C_WIDTH-1:0] c
);

    // Multiplier
    always_ff @(posedge clk or negedge resetn) begin
        if (!resetn) begin
            c <= 64'h0;
        end else begin
            c <= a * b;
        end
    end
endmodule
```

This module takes 32-bit inputs `a` and `b` and outputs a 64-bit `product`. The multiplication operation is performed using the built-in `*` operator in Verilog. The product register is updated on the rising edge of the `clk` signal and can be reset using the asynchronous active-low `resetn` signal[8].

A Short Introduction to SystemVerilog[a]

`SystemVerilog` is an advanced hardware description language and a superset of `Verilog`. It is the latest version, incorporating all `Verilog` features and adding new constructs for both design[b] and verification[c]. `SystemVerilog` enhances design abstraction, supports object-oriented programming, and provides advanced constructs for testbenches and verification, making it the preferred choice for complex digital systems.

In `Verilog`, there are two primary data types used to model digital signals: `reg` and `wire`. SystemVerilog introduced a new data type called `logic` to address some limitations of the original Verilog data types.

- In Verilog, `reg` is short for "register" and represents a storage element in `Verilog`. It can model sequential and combinational logic, such as flip-flops and procedural assignments (e.g., within always blocks). It is important to note that `reg` does not necessarily

[8] An excellent paper explaining the difference between synchronous reset and asynchronous reset: `http://www.sunburst-design.com/papers/CummingsSNUG2003Boston_Resets.pdf`

mean a physical register in the hardware; it is just a term used for data storage in the language.

- In `Verilog`, `wire` is a data type used to model connections between elements, such as gates or modules. Wires are used for continuous assignments and do not store values. They represent physical connections in a digital circuit, and the output of connected elements drives their values.

- In `SystemVerilog`, the `logic` data type is introduced to address the limitations of `reg` and `wire`. The `logic` type can be used for both storage (like `reg`) and continuous assignments (like `wire`). This makes it a more versatile and less error-prone data type than using `reg` and wire separately.

The primary differences between `logic` and `reg/wire` are:

- `logic` can be used for both procedural and continuous assignments, whereas `reg` is used for procedural assignments, and `wire` is used for continuous assignments.

- `logic` can be used as an `input`, `output`, or `inout` port of a module, while `wire` can only be used for `input` or `inout` ports, and `reg` is not directly used as a port type.

- `logic` supports four-state values (`0`, `1`, `X`, and `Z`) like `reg` and `wire`, but it can also be used for packed arrays[d] and structures[e], making it more suitable for complex data types.

In short, the `logic` data type in SystemVerilog offers a more flexible and unified way to represent digital signals compared to `reg` and `wire` in Verilog. It simplifies the code and reduces the chances of errors due to data type mismatches.

Moreover, you must have noticed the `always_ff` block introduced in SystemVerilog to explicitly model clocked sequential logic (e.g., flip-flops or registers). The `always_ff` block has a more restrictive sensitivity list than the `always` block: it must contain only edge-sensitive events. Similarly, SystemVerilog also introduces the `always_comb` block that is specifically designed for modeling combinational logic. It automatically generates the appropriate sensitivity list, making it easier to write and maintain combinational logic compared to using the `always` block in Verilog.

By using `always_ff` and `always_comb`, the designer's intent is clearer, and the potential for errors in the code is reduced.

[a] To know more new features of `SystemVerilog` over `Verilog`, read: `http://www.sunburst-design.com/papers/CummingsSNUG2003Boston_SystemVerilog_VHDL.pdf`

[b] `SystemVerilog code for digital hardware design: https://sutherland-hdl.com/papers/2013-SNUG-SV_Synthesizable-SystemVerilog_paper.pdfAgreatbookonwritingsynthesizable\protect\unhbox\voidb@x\bgroup\edefSystemVerilog`. ,.,,

[c] If you are also interested in verification: `https://www.amazon.com/RTL-Modeling-SystemVerilog-Simulation-Synthesis/dp/1546776346`

[d] `SystemVerilog` Packed Arrays: `https://www.chipverify.com/systemverilog/systemverilog-packed-arrays`

[e] `SystemVerilog` Structures: `https://www.chipverify.com/systemverilog/systemverilog-structure`

We have prepared a SystemVerilog testbench `tb_mul.sv` under `~/lab1/tut2/src/testbench`. The following testbench code verifies the multiplier on all test cases in a nine-nine table (from 1×1

to 9×9).

```
`timescale 1ns/1ps

module tb_multiplier_32bit();
    // Local parameters
    localparam CLK_PERIOD    = 10;
    localparam APPLY_DELAY   = CLK_PERIOD/2;
    localparam RECEIVE_DELAY = 3*CLK_PERIOD/4;
    localparam A_WIDTH       = 32;
    localparam B_WIDTH       = 32;
    localparam C_WIDTH       = 64;

    // Instantiate Signals
    logic        clk;
    logic        resetn;
    logic [31:0] a;
    logic [31:0] b;
    logic [63:0] c;

    // Instantiate the multiplier (Design Under Test)
    multiplier_32bit dut (
        .clk    (clk),
        .resetn (resetn),
        .a      (a),
        .b      (b),
        .c      (c)
    );

    // Clock generation
    always begin
        #(CLK_PERIOD/2) clk = ~clk;
    end

    // Apply the stimulus to the multiplier
    task apply_stimulus;
        // Test all combinations in the nine-nine table
        for (a = 1; a <= 9; a = a + 1) begin
            for (b = 1; b <= 9; b = b + 1) begin
                @(posedge clk);  // Wait until the next posedge of clk
                #APPLY_DELAY;
            end
        end
    endtask

    // Receive the responses from the multiplier
    task receive_responses;
        logic [31:0] buf_a;
        logic [31:0] buf_b;
        @(posedge clk); // Receive the response one clock cycle later
        for (int unsigned i = 1; i <= 9; i = i + 1) begin
            for (int unsigned j = 1; j <= 9; j = j + 1) begin
                buf_a = a;
                buf_b = b;
```

```
                $write("a = %d, b = %d, ", buf_a, buf_b);
                #RECEIVE_DELAY $display("c = %d", c);
                if (c !== buf_a * buf_b) begin
                    $display("Error: Incorrect result for %d x %d", buf_a, buf_b);
                end
                @(posedge clk);  // Wait until the next posedge of clk
            end
        end
    endtask

    // Testbench stimulus
    initial begin
        // Initialize signals
        clk = 0;
        resetn = 1;
        a = 0;
        b = 0;

        // Reset the multiplier
        @(posedge clk);
        #APPLY_DELAY resetn = 0;
        @(posedge clk);
        #APPLY_DELAY resetn = 1;

        // Run tasks in parallel
        fork
            apply_stimulus;
            receive_responses;
        join

        // Finish the test
        $display("Testbench completed.");
        $finish;
    end
endmodule
```

Next, navigate to the `questasim` folder to compile the code and launch the simulation:

```
cd ~/lab1/tut2/questasim
vlog -sv ../src/design/mul.sv ../src/testbench/tb_mul.sv
vsim -voptargs=+acc tb_multiplier_32bit
```

- The `vlog`[9] command is used to compile `SystemVerilog` and `Verilog` source files. In this case, the command compiles two files: `mul.sv` (the design file) and `tb_mul.sv` (the testbench file). The `-sv` flag tells the compiler to treat the input files as SystemVerilog source files. This flag is used when you have SystemVerilog constructs in your code that are unavailable in standard Verilog.
- The `vsim`[10] command is used to launch the QuestaSim simulator with the specified testbench. In this case, the testbench module is `tb_multiplier_32bit`. The `-voptargs=+acc` flag

[9] To know more about `vlog`, type `vlog -help all`
[10] To know more about `vsim`, type `vsim -help all`

passes the `+acc` option to the `vopt` optimization and elaboration tool, which is used during the simulation process. The `+acc` option instructs `vopt` to provide full access to all the design hierarchy and signals, making them available for viewing and probing during simulation. This option is useful when observing and debugging internal signals in your design. However, when simulating a very large design without needing to view every signal, you should not use `-voptargs=+acc` to turn on optimization and make your testbench run faster.

Next, in the `QuestaSim` GUI, type the following commands in the command-line window to create a wave window, add signals and run the simulation:

```
view wave
add wave -recursive tb_multiplier_32bit/*
run -all
```

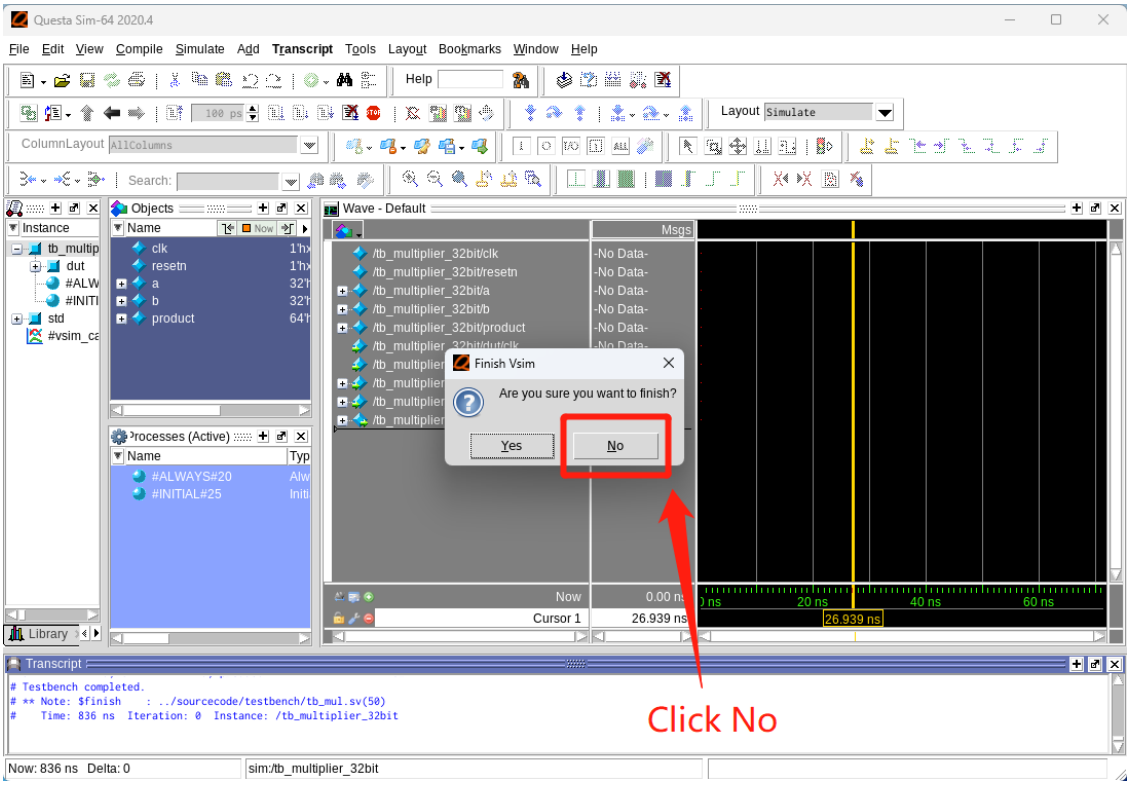

Figure 6: Simulation in `QuestaSim`

You will see a pop-up window once the simulation ends, as shown in Figure 6. Click `No` so that `QuestaSim` does not close, and you can view the waveform of signals. To zoom in or zoom out your wave window, press `i` and `o`.

- The `view wave` command opens a new wave window in QuestaSim. The wave window is a graphical interface for viewing and analyzing signal waveforms during simulation.
- `add wave -recursive tb_multiplier_32bit/*` adds all the signals in the design hierarchy under the `tb_multiplier_32bit` testbench instance to the wave window. The `-recursive` flag ensures that all signals at all levels of hierarchy within the testbench instance are added to the wave window.

- `run -all` command runs the simulation until it reaches the end or encounters a `$finish` or `$stop` statement in the testbench. The `-all` flag means the simulation will continue running until there are no more events to process.

Next, type in the following command:

```
wave zoom full
```

`wave zoom full` command adjusts the wave window's zoom level to display the full range of time covered by the simulation. It lets you see all the signal waveforms from the simulation's start to end in a single view. Alternatively, you can click on the wave window and press `f`.

After you inspect the waveforms, **please close the Questa Sim window** before continuing to the next section.

4.2 Design Automation using `Bash Scripts`

Repeatedly typing commands or clicking buttons on a GUI can be tedious and time-consuming. However, with the help of `Bash Scripts`, we can automate various tasks such as simulation, synthesis, place-and-route, and many others, streamlining our workflow and saving valuable time.

The `Bash Script` is a popular scripting language used in Unix-based operating systems to automate tasks such as executing commands, performing file operations, and managing system processes. It's versatile, ranging from simple one-liners to complex scripts, making it valuable for both beginners and advanced users. Its simple syntax and wide range of features make it a powerful tool for automating repetitive tasks and solving specific problems.

To make your life easier, we have put all the commands you typed above in a `Bash Script` `tb_mul.sh` under `~/lab1/tut2/questasim`. Run the script in a terminal by:

```
source tb_mul.sh
```

Take a moment to inspect the `tb_mul.sh` script, and you will notice some differences:

- The line `#!/bin/bash` at the start of a `Bash Script` is called a `shebang`[11]. It specifies the interpreter, in this case, the 'bash' shell, to run the script on a Unix-based system. The shebang is important as it tells the system how to interpret the script, ensuring it runs correctly.

- The `vlib` command creates a new work library. The `vmap` command maps the work library to the name `workLib`, allowing it to be used in the simulation. In `Verilog` simulations, work libraries store compiled design files and simulation results. By creating a work library, the script ensures that all the necessary design files are compiled and ready for simulation. The `vmap` command maps the work library to a name, making referencing it in other parts of the script easier.

- In the `vsim` command, The `-do` option specifies a simulation command file `tb_mul.cmd` to run the simulation. The `-t` option sets the simulation time resolution limit to 100 ps.

[11] `https://en.wikipedia.org/wiki/Shebang_(Unix)`

5 Tutorial 3: Compiling RISC-V C Programs

5.1 Step-by-Step

In this tutorial, you will learn how to use the RISC-V toolchain to compile a simple C program.

5.1.1 Step 1: Write an Embedded C Program

Navigate to the Tutorial 3 folder `tut3`:

```
cd ~/lab1/tut3
```

Inspect the firmware C program `sum.c` located under `~/lab1/tut3/firmware`:

```
void main() {
    int a = 1;
    int b = 2;
    int c = a + b;
}

/*
 * Define the entry point of the program.
 */
__attribute__((section(".text.start")))
void _start()
{
    main();
}
```

- `void _start(void)` is a custom entry point function for the program. Typically, the entry point function initializes the runtime environment before calling the `main()` function. In this example, the `_start()` function calls the `main()` function.
- The `_start()` function is decorated with a GCC attribute `__attribute__((section(".text.start")))`, which places the function in a specific section named `.text.start`. This is useful for ensuring that the `_start()` function appears at the beginning of the `.text` section when linked with a linker script that has the appropriate input section pattern (e.g., `*(.text.start .text.start.*)`), which you will see later.

In short, when the program is executed, the `_start()` function will be called first because it is defined as the program's entry point. The `_start()` function, in turn, calls the `main()` function, which then performs the simple arithmetic operations as described earlier.

5.1.2 Step 2: Preparation Before Compilation

Now we can compile the C program with the `GNU GCC` cross-compiler for RISC-V ISA, basically using the `riscv32-unknown-elf-gcc` command, but how can we locate the executable linked to this command?

Entering a command on the terminal instructs the shell to execute an executable file with the specified name. In Linux, these executable programs, such as `ls` and `find`, are typically found in various directories on the system. Any file stored in these directories with executable permission can be run from any location. The directories most commonly containing these executables are `/bin`,

`/sbin`, `/usr/sbin`, `/usr/local/bin`, and `/usr/local/sbin`. But how does the shell determine which directories to search for these executables? Does it examine the entire file system?

The answer is simple. When a command is entered, the shell searches for the executable file with the specified name in all directories listed in the user's `$PATH` variable.

To correctly compile our applications, we must add the path to the RISC-V compiler to the `$PATH` variable. The executable of command `riscv32-unknown-elf-gcc` is located at `/data/picorv32-utils/riscv32imc/bin`. This can be achieved by executing the following command:

```
1  echo "export PATH=\"/data/picorv32-utils/riscv32imc/bin:\$PATH\"" >> ~/.bashrc
2  source ~/.bashrc
```

- The `echo` command prints the string and `>> ~/.bashrc` appends the output of the `echo` command to the `.bashrc` file in your home directory. If the file doesn't exist, it will be created.
- `source ~/.bashrc` loads the changes you made to the `.bashrc` file (don't miss the dot `.`), effectively updating your `$PATH` variable.

5.1.3 Step 3: Compile the Code

Now, let us compile the C code:

```
1  cd ~/lab1/tut3/firmware
2  riscv32-unknown-elf-gcc -S sum.c -o sum.s -march=rv32imc
3  riscv32-unknown-elf-gcc -c sum.s -o sum.o
```

- The `-S` option tells GCC to generate an assembly file. When using this flag, the compiler will take a source code file (e.g., a C or C++ file) and produce the corresponding assembly code in a `.s` file. The generated assembly code can be helpful for debugging, optimization, or learning purposes.
- The `-c` option tells GCC to generate an object file. When using this flag, the compiler will take source code files and produce the corresponding object code in a `.o` file.
- The `-o` option specifies the name of the output file.
- The `-march` option specifies the target architecture and the set of architectural features the generated code should support. It is followed by a string representing the desired RISC-V architecture variant. In the case of `-march=rv32imc`, the option specifies a RISC-V 32-bit architecture with the following extensions:
  - `rv32` indicates that the base architecture is RISC-V 32-bit.
  - `i` represents the base integer instruction set, which includes basic arithmetic and logic operations, branches, and loads/stores for 32-bit integer data types.
  - `m` adds support for integer multiplication and division instructions.
  - `c` stands for "Compressed," and it supports 16-bit compressed instructions, which can help reduce code size and improve energy efficiency.

  Choosing the correct `-march` option is essential to ensure that your compiled code runs correctly on your target RISC-V processor, in our case, the `PicoRV32` core supporting `rv32imc`.

5.1.4 Step 4: Generate the Executable

First, let us create a very basic linker script `linkerscript.ld`, which is provided under `lab1/tut3/firmware` for your convinience:

```
ENTRY(_start)

SECTIONS
{
  .text : {
    *(.text.start .text.start.*)
    *(.text)
  }

  .data : {
    *(.data)
  }

  .bss : {
    *(.bss)
  }
}
```

- The `ENTRY(_start)` directive specifies the entry point of the program, which is the symbol `_start`. The entry point is the function that will be executed first when the program starts running.
- The `SECTIONS` command is used to define the layout of the output file. It contains a series of section definitions, which tell the linker how to arrange the various sections in the resulting binary.
- `.text` section is used for storing the executable code of the program. The following input section patterns are used to populate the '.text' section in the output file:
  - The `*(.text.start .text.start.*)` section pattern gathers all input sections with the names `.text.start` and `.text.start.*`. These sections typically contain the entry point function `_start` and other initialization code that must be executed before the main program.
  - The `*(.text)` section pattern gathers all input sections with the name `.text`. These sections contain the main executable code of the program.
- The `.data` section is used for storing initialized global and static variables.
- `.bss`: This section stores uninitialized global and static variables, which are initialized with zero in the startup code. It occupies no space in the output file but is allocated at runtime.

This linker script provides a simple layout for an executable program, with separate sections for code, initialized data, and uninitialized data. It ensures that the `_start` function is placed at the beginning of the `.text` section will be the first function to execute when the program starts.

Now, we can generate the executable using the Linker with our linker script:

```
riscv32-unknown-elf-ld sum.o -o sum.elf -T linkerscript.ld
```

- `riscv32-unknown-elf-ld` is the RISC-V linker command. It is a version of the GNU linker (ld) tailored to work with RISC-V 32-bit architectures.
- `sum.o` is the input object file that you want to link.
- The `-o sum.elf` option specifies the output file name for the linker. In this case, the output file will be named sum.elf.
- The `-T linkerscript.ld` option tells the linker to use the provided linker script linkerscript.ld to control the linking process. The linker script defines the memory layout, entry point, and organization of sections in the output file.

The resulting sum.elf file will be an executable binary in ELF format, built for the RISC-V 32-bit architecture and organized according to the rules defined in the linkerscript.ld.

5.2 Compilation Automation using `Makefile`

Similar to how you have automated the design process using Bash scripts, Makefiles can help you streamline and automate the code compilation, linking, and even testing processes for your projects. By defining targets and their dependencies in a Makefile, you can efficiently manage the build process, ensuring that only the necessary files are recompiled when changes are made, saving time and resources.

A Short Introduction to Makefile[a]

A `Makefile` is a script used by the `make` utility to automate the process of building, compiling, and managing dependencies in software projects. `Makefile` are particularly useful for large projects with many files and dependencies, as they help simplify and streamline the build process.

`Makefile` consist of a series of rules, each specifying a target, its dependencies, and a set of commands to create or update the target. Here's a general overview of how `Makefile` work:

- **Target**: A target is an entity that `make` creates or updates. It can represent a file, such as an executable, object file, or library, or it can be a phony target, which doesn't represent a file but serves as a way to group other targets or commands.
- **Dependencies**: Dependencies are files or other targets that must be up-to-date before the target can be built. If any of the dependencies are newer than the target, the target will be considered out-of-date and will be rebuilt.
- **Commands**: Commands are a series of shell commands that `make` runs to create or update the target. Commands are associated with a target and are executed only if it needs to be updated (i.e. if it is out-of-date or doesn't exist).
- **Variables**: Makefiles can include variables, which allow you to store and reuse values. Variables can be used to define paths, compiler flags, or any other values you want to reference throughout the `Makefile`.

- **Automatic Variables**: Automatic variables are predefined variables that provide useful information about the current target, its dependencies, and other aspects of the build process. Some common automatic variables include `$@` (the target name), `$<` (the first dependency), and `$^` (the list of all dependencies).

- **Phony Targets**: A phony target is a special kind of target that doesn't represent a file. Phony targets are useful for grouping other targets, running commands that don't produce a file, or specifying targets that should always be executed, even if a file with the same name exists.

[a] To learn more `Makefile` examples, explore: `https://makefiletutorial.com/`

Before we run the Makefile, navigate to the `~/lab1/tut3/firmware` directory and run the following command to clean the files compiled in the previous tutorial section:

```
make clean
```

Now, let's run the Makefile to complete the whole embedded C code compilation process above with a single command:

```
make
```

When you run the `make` command, it reads the `Makefile` in the current directory, and by default, it tries to build the first target specified in the `Makefile`. You can also specify a target to build by providing it as an argument to the `make` command (e.g., `make clean`).

The `make` utility checks the dependencies of the specified target. If any dependency is newer than the target or the target doesn't exist, it executes the associated commands to build the target. If the target has dependencies that are also targeted, `make` recursively checks and updates them as needed before building the main target.

Now, if we inspect the `Makefile`:

```
CROSS = /data/picorv32-utils/riscv32imc/bin/riscv32-unknown-elf-
CFLAGS = -O0

sum.elf: sum.c linkerscript.ld
  $(CROSS)gcc $(CFLAGS) -march=rv32imc \
    -T linkerscript.ld \
    -nostdlib \
    -o $@ \
    $<

clean:
  rm -f sum.elf

.PHONY: clean
```

- The first line defines the `CROSS` variable, which specifies the path to the RISC-V cross-compilation toolchain. This toolchain is used to build the project for the RISC-V 32-bit architecture with the IMC (Integer Multiplication and Compression) extension.

- The second line defines the `CFLAGS` variable, which stores the compiler flags. Here, the flag `-O0` is used to specify that the compiler should not optimize the code (optimize for debugging

purposes)[12].

- The `sum.elf: sum.c linkerscript.ld` line defines the `sum.elf` **target**, which depends on the `sum.c` and `linkerscript.ld`. The following commands in the indented block are used to build the `sum.elf` target using the GCC cross-compiler and calling the linker internally:
  - The `$(CROSS)gcc $(CFLAGS) -march=rv32imc` line calls the RISC-V cross-compiler `$(CROSS)gcc` with the specified compiler flags ($(CFLAGS)), and sets the target architecture to RISC-V 32-bit with the IMC extension (-`march=rv32imc`).
  - The `nostdlib` option tells the compiler not to use the standard libraries[13] during compilation.
  - The `-o $@` option specifies the output file name, where `$@` refers to the target name (`sum.elf`).
  - `$<` is an automatic variable that refers to the first dependency (`sum.c`).
- `clean:` defines the clean target, which doesn't have any dependencies. The following command in the indented block is used to clean the build artifacts:
- `.PHONY: clean` tells `make` that `clean` is a phony target, which means it doesn't represent a file and should always be executed, even if there is a file with the same name.

[12] GCC optimization options: `https://developer.arm.com/documentation/den0013/d/Optimizing-Code-to-Run-on-ARM-Processors/Compiler-optimizations/GCC-optimization-options`

[13] `https://en.cppreference.com/w/c/header`

6 Task 1: Hello World

In this task, you will learn how to simulate the `PicoSoC`, in which the `PicoRV32` core runs the "Hello, World!" C program.

Ensure you are in the Task 1 directory of Lab 1:

```
cd ~/lab1/task1
```

Compile `hello.c` code using the RISC-V cross-compiler:

```
cd firmware
make hello
```

Run simulation in QuestaSim:

```
cd ~/lab1
source setup.sh
cd  ~/lab1/task1/questasim
source task1.sh
```

You will see "Hello, World!" printed on the terminal.

7 Task 2: Dummy Accelerator

The dummy accelerator, declared in `~/lab1/task2/src/design/accelerator.v`, features an internal counter that increments in response to an input value. Upon reaching the target value, the counter value is assigned to the output register, and the 'DONE' flag is raised, indicating completion.

Ensure you are in the Task 2 directory of Lab 1:

```
cd ~/lab1/task2
```

Compile `dummy_accel.c` code using the RISC-V cross-compiler:

```
cd firmware
make dummy_accel
```

Run simulation in QuestaSim:

```
cd ../questasim
source task2.sh
```

You will see 'Accelerator Output: 0x00012345' printed on the terminal.

8 Task 3: Fast-Fourier Transform Algorithm

For this year's edition of the course, you will be working with Fourier transforms. A Fourier transform is a mathematical operation that converts a continuous time-domain signal ($x(t)$) into the frequency domain ($X(f)$):

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-j2\pi ft}\, dt \tag{1}$$

.

where f is the frequency, t is the time and j indicates that the exponent is a complex number. Fourier Transforms are one of the key building blocks of many scientific and engineering applications: the use cases of Fourier Transforms span from signal processing (filtering, noise reduction), image processing (compression like JPEG), communication systems (modulation) and quantum mechanics to solving partial differential equations. Due to this widespread adoption, it is very likely that already several thousand Fourier Transforms were computed by your laptop or phone just this morning.

Considering the formulation of (1) for the operation, it is clear that a continuous time domain input signal is required. While real-world signals are indeed continuous, our computing devices mostly operate in the discrete domain (that is, on data sampled from a continuous signal). Therefore, in order to compute this transformation in practice, a discrete formulation is required. The algorithm to do this is referred to as the Discrete Fourier Transform (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j\frac{2\pi}{N}kn}, \qquad k = 0, 1, \ldots, N-1 \tag{2}$$

.

This equation closely mirrors (1), but replaces the continuous integral with a finite-length sum. Unfortunately, this algorithm has a time complexity of $\mathcal{O}(n^2)$: for each of the N inputs, a sum of N multiplications has to be computed. This complexity becomes especially prohibitive for longer signals. Lucky for us, in 1965 James Cooley and and John Tukey re-discovered a much more efficient way to implement this Discrete Fourier Transform [1]. This approach was first pioneered by Carl Friederich Gauss in 1805, but never officially published. By leveraging properties of the complex coefficients in (2), namely their symmetry ($e^{-j\frac{2\pi}{N}\left(\frac{k+N}{2}\right)} = e^{-j\frac{2\pi}{N}(-k)}$) and periodicity ($e^{-j\frac{2\pi}{N}k} = e^{-j\frac{2\pi}{N}(k+N)}$), it is possible to reduce the complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(N\log(N))$ by applying a divide and conquer approach as seen in the figure below[14][15]:

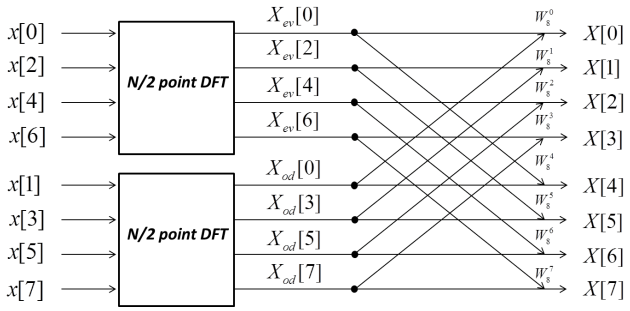

Figure 7: Decimation-in-Time FFT algorithm, where $W_N^{kn} = e^{-j2\pi/N \cdot kn}$

This is the algorithm that is referred to as the Fast Fourier Transform (FFT). If you are interested in a rigorous mathematical workout from DFT to FFT, please refer to 10. For this lab, we provide an example code (`fft.c`) that implements the FFT algorithm to obtain the frequency response of a signal, as shown in Figure 8. The signal samples, each occupying 4 bytes, are stored in external flash memory, as illustrated in Figure 9.

[14] `https://vocal.com/noise-reduction/fft-algorithms/`

[15] `https://vanhunteradams.com/FFT/FFT.html` - For a full in-depth explanation of FT and its variants

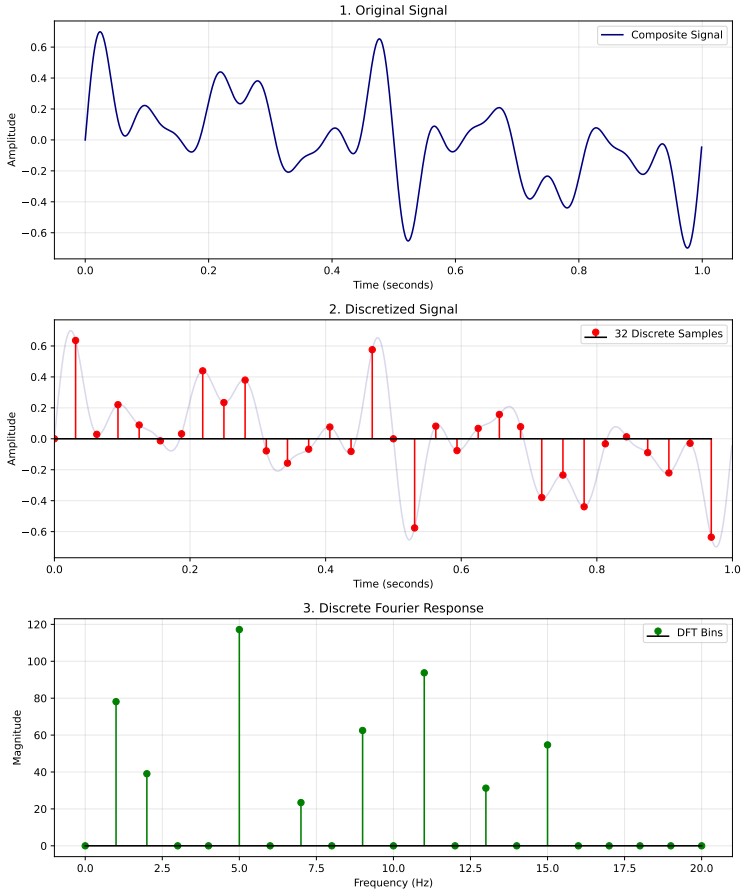

Figure 8: Example of frequency decomposition using FFT.

These samples are placed at the end of the flash, starting from address `0x004F0000`. Given that the flash memory extends up to `0x004FFFFF`, a total of 65,536 bytes is available for storage.

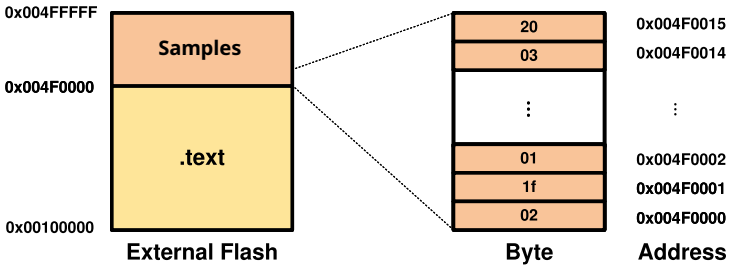

Figure 9: Flash Memory Map of Signal Samples.

Ensure that you are in the Task 3 directory of Lab 1:

```
cd ~/lab1/task3
```

Compile `fft.c` code using the RISC-V cross-compiler:

```
cd firmware
make clean
```

```
make
```

Run simulation in QuestaSim:

```
cd ../questasim
source task3.sh
```

To verify whether the code has done the calculation correctly:

```
python verify.py
```

The objective of your project is to develop an accelerator capable of implementing optimized versions of the FFT algorithm. Investigate different implementation approaches and other design strategies involving memory hierarchy and accesses. The accelerator should demonstrate a considerable reduction in latency when compared (1) to a RISC-V core, even with `-O2` compilation optimization, and (2) to the Cooley-FFT baseline that will be provided to you in Lab 2. More information on the project will come later.

9 Questions

You should submit a single report addressing questions for each of the three labs. The report template is available on BrightSpace. We will provide you with feedback after you submit the lab report for Lab 3.

Still, you have the freedom to submit your intermediate report during Week 1∼3 if you want to get feedback earlier on Lab 1 & 2. If you choose to do so, we will not give feedback on Labs 1 & 2 questions again when you submit your final lab report.

For this specific lab, please provide individual answers to the following questions, as they may be related to potential questions in your written examination:

1. For Task 1, please provide a detailed, step-by-step explanation of the testbench's operation. Specifically, answer:
   (a) How does `PicoSoC` load the firmware? (Hint: Check `spiflash.v`)
   (b) How does `PicoSoC` display the "Hello, World!" message on `QuestaSim`'s command line? Basically, explain to which memory address the ASCII[16] characters are written by the `print_str` function in `hello.c` and how `PicoSoC` sends out the characters to the testbench through the `ser_tx` port. (Hint: Check `simpleuart.v`, `picosoc.v`, `uart.c`, `uart.h` and other related files.)
2. For Task 2, carefully examine the `Verilog` code for the dummy accelerator and explain its functionality. How is the accelerator launched? How does it send its output to the RISC-V core with a handshake for robust data exchange?
3. In Task 2, closely inspect the `dummy_accel.c` file and provide a thorough explanation (by annotating the relevant C and Verilog code) of how the `PicoRV32` core interacts with the dummy accelerator, such as sending inputs to and receiving outputs from the accelerator.
4. In Task 3, closely inspect the Verilog code for the flash memory (`spiflash.v`) and explain briefly how the twiddle factors and samples are stored in the flash memory.
5. In Task 3, adjust the `$CFLAGS` variables using various optimization levels, `-O0`, `-O1` or `-O2` and report the resulting latency in clock cycles.

[16] `https://www.asciitable.com/?utm_content=cmp-true`

10 From DFT to FFT in detail

The discrete Fourier transform (DFT) of a sequence $x[n]$ of length N is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad W_N = e^{-j2\pi/N}, \quad k = 0, \ldots, N-1. \tag{3}$$

Here, W_N is known as twiddle factor. It exhibits several important properties that allow us to reduce the computational complexity of the DFT.

One such property is periodicity:

$$W_N^{n+N} = W_N^n \tag{4}$$

This means that powers of W_N repeat every N steps. Another key property is symmetry:

$$W_N^{2m} = W_{N/2}^m \tag{5}$$

This allows us to relate the twiddle factors of length N to those of length $N/2$, which will be useful when we split the DFT into even- and odd-indexed terms.

We can split the original sum into contributions from even and odd indices:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N/2-1} x[2n] W_N^{k(2n)} + \sum_{n=0}^{N/2-1} x[2n+1] {W_N}^{k(2n+1)} \\ &= \sum_{n=0}^{N/2-1} x[2n] W_{N/2}^{kn} + W_N^k \sum_{n=0}^{N/2-1} x[2n+1] W_{N/2}^{kn}. \end{aligned} \tag{6}$$

At this point, it is convenient to define two separate sequences corresponding to the even and odd terms:

$$\begin{aligned} E[k] &= \sum_{n=0}^{N/2-1} x[2n] W_{N/2}^{kn} \\ O[k] &= \sum_{n=0}^{N/2-1} x[2n+1] W_{N/2}^{kn} \end{aligned} \tag{7}$$

Using the periodicity property of W_N, we can express the DFT of the full sequence in terms of these half-length sequences (full work out starts with 9):

$$\begin{aligned} X[k] &= E[k] + W_N^k O[k] \\ X[k + N/2] &= E[k] - W_N^k O[k], \end{aligned} \qquad k = 0, \ldots, N/2 - 1. \tag{8}$$

This shows that by computing $E[k]$ and $O[k]$ for the first half of the sequence ($k = 0, \ldots, N/2 - 1$), we can immediately reuse these results to obtain the DFT values for the second half. Each of these half-length sequences can then be recursively split in the same way until we reach sequences of length 1, at which point the DFT is trivial to compute. This recursive splitting leads to $\log_2 N$ stages, each requiring N operations, reducing the overall computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$.

To see why $E[k]$ and $O[k]$ repeat for the second half of the sequence, consider $E[k + N/2]$:

$$\begin{aligned} E[k + N/2] &= \sum_{n=0}^{N/2-1} x[2n] \left(W_{N/2}^{k+N/2}\right)^n \\ &= \sum_{n=0}^{N/2-1} x[2n] \left(W_{N/2}^{k}\right)^n \\ &= \sum_{n=0}^{N/2-1} x[2n] W_{N/2}^{kn} \\ E[k + N/2] &= E[k] \end{aligned} \tag{9}$$

The same applies to the odd-indexed sequence:

$$\begin{aligned} O[k + N/2] &= \sum_{n=0}^{N/2-1} x[2n+1] \left(W_{N/2}^{k+N/2}\right)^n \\ &= \sum_{n=0}^{N/2-1} x[2n+1] \left(W_{N/2}^{k}\right)^n \\ &= \sum_{n=0}^{N/2-1} x[2n+1] W_{N/2}^{kn} \\ O[k + N/2] &= O[k] \end{aligned} \tag{10}$$

Finally, the twiddle factor W_N^k changes sign in the second half of the sequence due to periodicity:

$$\begin{aligned} W_N^{k+N/2} &= \left(e^{-j2\pi/N}\right)^{k+N/2} \\ &= \left(e^{-j2\pi/N}\right)^{k} \cdot \left(e^{-j2\pi/N}\right)^{N/2} \\ &= e^{-j2\pi k/N} \cdot e^{-j\pi} \\ &= e^{-j2\pi k/N} \cdot -1 \\ W_N^{k+N/2} &= W_N^k \cdot -1 \\ W_N^{k+N/2} &= -W_N^k. \end{aligned} \tag{11}$$

This explains the minus sign in the second half of the DFT computation in the recursive algorithm.

References

[1] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965.